

MECHATRONIC SYSTEM INTEGRATION
MCTA 3203

EXPERIMENT 2 :
SERIAL COMMUNICATION BETWEEN ARDUINO & PYTHON

SECTION 2
SEMESTER 1, 2026/2027

INSTRUCTOR :
DR. AHMAD IMRAN BIN IBRAHIM
DR. ING. WAHJU SEDIONO
TS. DR. YASIR BIN MOHD MUSTAFAH

DATE OF SUBMISSION : 17 OCTOBER 2026

GROUP 9

NO.	NAME	NO. MATRIC
1	MUHAMMAD DANISH FITRI BIN MOHD SHARIFUDDIN	2413559
2	MUHAMMAD IZZ ZIKRY BIN AZWARI	2218899
3	SAMER Y. I. LUBBAD	2213959
4	MUHAMMAD FARIS IZHAN BIN JOHARI	2413501

ABSTRACT

The main objective of this experiment was to design a two-step smart access control system by integrating an MPU6050 motion sensor and a USB RFID reader with an Arduino Uno. The project focused on establishing a stable two-way serial communication link between the physical hardware and a Python script. Python was utilized to manage the RFID scanner in a background thread and to analyze the continuous X and Y acceleration data from the sensor. By calculating the data variance and using cross-product math, the Python code successfully filtered out random vibrations to accurately identify intentional circular gestures. After an authorized RFID card was scanned and the circular motion was verified, Python sent a byte command to the Arduino to unlock the system. The Arduino acted strictly as the physical actuator. It used a non-blocking timer to move a servo motor and switch the LED status lights without interrupting the live sensor data stream. Overall, the experiment successfully proved that complex mathematical processing can be offloaded from a basic microcontroller to a computer to create a highly responsive, gesture-based security system.

TABLE OF CONTENTS

1.0 INTRODUCTION....4
1.1 MATERIALS & EQUIPMENTS....4
1.2 EXPERIMENTAL SETUP....5
1.2.1 Circuit Setup....5
1.2.2 Circuit diagram....5
1.3 METHODOLOGY & PROGRAMMING LOGIC....6
1.3.1 Experiment Steps....6
1.3.2 Description of Motion Detection Approach....6
1.3.3 Integration Explanation (How Parts Work Together)....6
1.3.4 Arduino Code....7
1.3.5 Python Code....9
1.4 DATA COLLECTION & ANALYSIS....13
1.4.1 Data Collection....13
1.4.2 Data Analysis....13
1.5 RESULTS & DISCUSSION....15
1.5.1 Results....15
1.5.2 Discussion....15
1.6 CONCLUSION....16
1.7 RECOMMENDATIONS....16
REFERENCES....17
APPENDICES....17
ACKNOWLEDGEMENT....17
STUDENTS DECLARATION....18
Certificate of Originality and Authenticity....18

1.0 INTRODUCTION

Serial communication is a fundamental concept in modern mechatronics because it bridges the gap between simple microcontrollers and more powerful computers. This experiment explores how to combine multiple sensors to develop a two-step secure access system. By integrating a physical key card with gesture recognition, the project mimics real-world applications found in modern IoT security and Human-Machine Interfaces (HMI).

The core objective is to establish a two-way serial connection between the hardware and software. The Arduino Uno is programmed to continuously stream raw acceleration data from an MPU6050 sensor to a computer. At the same time, a Python script runs in the background waiting for input from a USB RFID reader. Instead of forcing the Arduino to process complex mathematics, the heavy lifting is offloaded to the computer. Python analyzes the sensor values in real-time to determine if the user is intentionally moving the sensor in a circular path.

The system operates on a dual-layer verification process. First, an authorized RFID card must be scanned to unlock the next stage. Once verified, the system waits for the correct circular motion. Only when both conditions are met does Python send a command back to the Arduino. This command moves a servo motor to simulate a physical lock and switches the LED status lights. Ultimately, this setup demonstrates how sensor data, real-time computer algorithms, and physical actuators can be combined to build a responsive mechatronic system.

1.1 MATERIALS & EQUIPMENTS

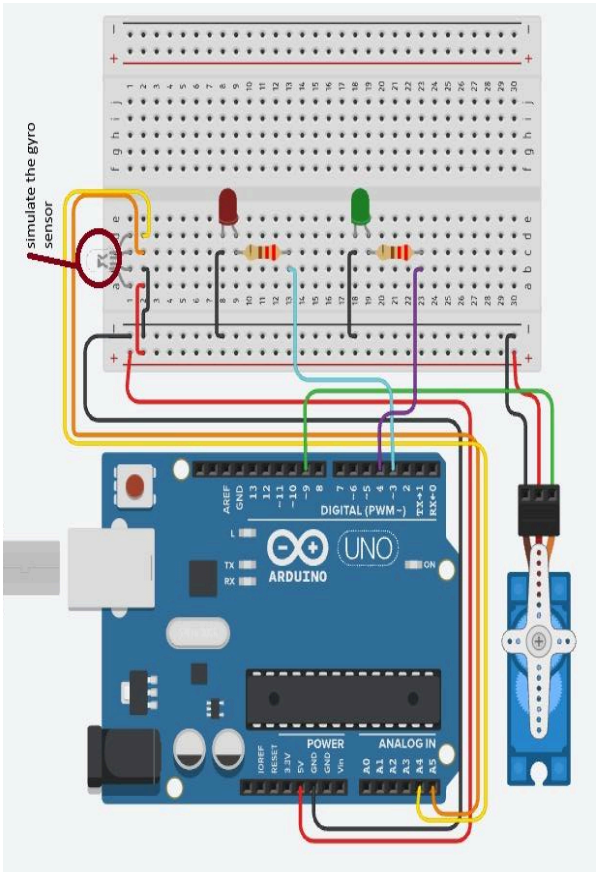
1. Arduino Uno board (1)
2. MPU6050 sensor (1)
3. Cytron R20D-USB EM4100 RFID Smart Card Reader (1)
4. Servo Motor (1)
5. 5mm LEDs (1 Red, 1 Green)
6. 220-ohm resistors (2)
7. Jumper wires & Breadboard
8. Laptop with PyCharm and Arduino IDE installed

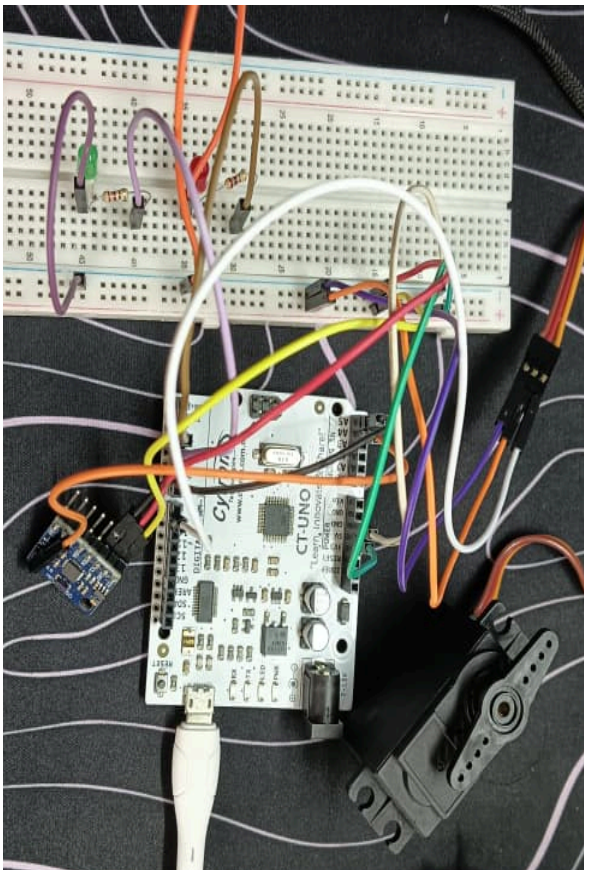
1.2 EXPERIMENTAL SETUP

1.2.1 Circuit Setup

1. **MPU6050:** The VCC pin was connected to 5V, GND to GND, SDA to A4, and SCL to A5.
2. **Servo Motor:** The signal wire was connected to digital pin 9. The power and ground wires were connected to 5V and GND.
3. **LEDs:** The long leg of the Red LED was connected to digital pin 3, and the long leg of the Green LED to digital pin 4. Both short legs were connected to GND using 220-ohm current-limiting resistors.
4. **RFID Reader:** This was plugged directly into the laptop via USB to act as a keyboard input.

1.2.2 Circuit diagram





1.3 METHODOLOGY & PROGRAMMING LOGIC

1.3.1 Experiment Steps

1. The electronic components were wired on the breadboard based on the circuit design.
2. The Arduino code with a millis() timer was uploaded to the board.
3. The Arduino IDE Serial Monitor was closed to avoid COM port connection errors.
4. The Python code was run in PyCharm to start reading the data and plotting the graph.
5. A specific RFID card was scanned using the USB reader.
6. The MPU6050 sensor was moved in a circle to trigger the Python code.
7. The LEDs and servo motor were observed to see if they worked correctly.

1.3.2 Description of Motion Detection Approach

The motion detection approach relies on analyzing the X and Y acceleration data sent from the MPU6050 to Python. A background program saves the last 40 sensor readings in a list. To figure out if the movement is a circle, the Python code does the following:

1. First, it centers the data to remove the constant pull of gravity.
2. Next, it checks the variance of the data to make sure the movement is large enough to ignore random vibrations.
3. It then compares the standard deviation of the X and Y axes. If the ratio is somewhat equal, it confirms the shape is round and not just a straight line.
4. Finally, it uses 2D cross-product math to check the direction of the points. If the points are continuously rotating in one direction (clockwise or counter-clockwise), the code officially detects a circular motion.

1.3.3 Integration Explanation (How Parts Work Together)

The system uses a two-factor verification process that splits the workload between the computer and the microcontroller.

- Python acts as the main brain. It runs a background thread that constantly listens for the USB RFID reader to type in a code without stopping the live data graph.
- When Python sees an authorized RFID scan, it unlocks the next stage and starts analyzing the MPU6050 sensor data using the motion detection approach described above.

- Once Python confirms the circular motion, it sends a single byte command ('R') over the serial port to the Arduino.
- The Arduino acts strictly as the physical controller. It runs a millis() state machine that listens for Python's command. When it receives 'R', it turns off the red LED, turns on the green LED, and moves the servo to 180°. The timer keeps the door open for exactly 5 seconds and then resets everything to the locked state automatically, while continuing to send sensor data back to Python the entire time.

1.3.4 Arduino Code

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo accessServo;

const int redPin = 3;
const int greenPin = 4;

// State Machine Variables
int currentState = 0;
// 0 = IDLE (Red ON, Servo at 0)
// 1 = WAITING_FOR_MOTION (Red pulsing, Servo at 0)
// 2 = UNLOCKED (Green ON, Servo at 180 for 5s)

unsigned long lastBlinkTime = 0;
bool blinkState = false;
unsigned long unlockTimer = 0;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  accessServo.attach(9);

  pinMode(redPin, OUTPUT);
  pinMode(greenPin, OUTPUT);
```

```
  // Initial state: Servo at 0, only Red LED on
  accessServo.write(0);
  digitalWrite(redPin, HIGH);
  digitalWrite(greenPin, LOW);

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
    while (1);
  }
}

void loop() {
  // 1. Send motion data to Python
  int ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(","); Serial.println(az);

  // 2. Listen for Serial Commands
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == 'W' && currentState == 0) {
      currentState = 1; // RFID scanned, wait for motion
    }
    else if (cmd == 'R' && currentState == 1) {
      currentState = 2; // Rotation detected, unlock
      unlockTimer = millis();

      accessServo.write(180);
      digitalWrite(redPin, LOW);
      digitalWrite(greenPin, HIGH);
    }
  }

  // 3. State Machine Logic (Non-blocking)
  unsigned long currentMillis = millis();

  if (currentState == 1) {
    // Handle Red LED Pulsing (Toggle every 250ms)
    if (currentMillis - lastBlinkTime >= 250) {
      lastBlinkTime = currentMillis;
      blinkState = !blinkState;
```

```
      digitalWrite(redPin, blinkState ? HIGH : LOW);
    }
  }
  else if (currentState == 2) {
    // 5-Second Hold Timer
    if (currentMillis - unlockTimer >= 5000) {
      currentState = 0; // Return to IDLE

      accessServo.write(0);
      digitalWrite(redPin, HIGH);
      digitalWrite(greenPin, LOW);
    }
  }

  delay(10); // Small delay to avoid serial buffer overflow
}
```

1.3.5 Python Code

```
import serial
import matplotlib.pyplot as plt
import numpy as np
from collections import deque
import threading
import time


def is_circular_motion(x_data, y_data):
   if len(x_data) < 10:
       return False
   x_centered = x_data - np.mean(x_data)
   y_centered = y_data - np.mean(y_data)
   if np.var(x_centered) < 10000 or np.var(y_centered) < 10000:
       return False
   std_x = np.std(x_centered)
   std_y = np.std(y_centered)
   if std_x == 0 or std_y == 0:
       return False
   ratio = std_x / std_y
   if ratio < 0.5 or ratio > 2.0:
       return False
   clockwise_turns = 0
   total_turns = len(x_data) - 1
   for i in range(total_turns):
```

```
        cross_product = (x_centered[i] * y_centered[i + 1]) - (y_centered[i] *
x_centered[i + 1])
        if cross_product > 0:
            clockwise_turns += 1
    turn_ratio = clockwise_turns / total_turns
    if turn_ratio > 0.70 or turn_ratio < 0.30:
        return True
    return False


def classify_motion(x_data, y_data, z_data, noise_threshold=3000):
    if is_circular_motion(x_data, y_data):
        return "Rotation (Circular) 🔄"
    std_x = np.std(x_data)
    std_y = np.std(y_data)
    std_z = np.std(z_data)
    max_std = max(std_x, std_y, std_z)
    if max_std < noise_threshold:
        return "Idle 🛑"
    if max_std == std_x:
        return "Left / Right ↔"
    elif max_std == std_y:
        return "Front / Back ↕"
    elif max_std == std_z:
        return "Up / Down ⬆️⬇️"
    return "Unknown 🤔"


# ================= RFID THREADING SETUP =================
# Updated with your specific USB RFID card code
AUTHORIZED_UIDS = ["0013020304"]
scanned_uid = None
system_state = "WAIT_RFID"
action_time = 0


def rfid_listener():
    """Background thread to catch the USB RFID keyboard emulation."""
    global scanned_uid
    while True:
        try:
            uid = input()
            if uid:
                scanned_uid = uid.strip()
        except:
            pass


threading.Thread(target=rfid_listener, daemon=True).start()

# Updated to COM5
ser = serial.Serial('COM5', 9600, timeout=1)
```

```
window_size = 40
history_x = deque(maxlen=window_size)
history_y = deque(maxlen=window_size)
history_z = deque(maxlen=window_size)

plot_x, plot_y = [], []

plt.ion()
fig, ax = plt.subplots()
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
line, = ax.plot([], [], 'ro')

print("\n--- SYSTEM LOCKED ---")
print("Make sure this PyCharm terminal is in focus.")
print("Tap your USB RFID card now...")

while True:
   try:
       current_time = time.time()

       # 1. Check if the 5-second door sequence has finished (giving it 6s to
be safe)
       if system_state == "DOOR_BUSY" and (current_time - action_time >=
6.0):
           system_state = "WAIT_RFID"
           print("\n--- SYSTEM LOCKED ---")
           print("Tap your USB RFID card now...")

       # 2. Check for new RFID scans
       if system_state == "WAIT_RFID" and scanned_uid is not None:
           if scanned_uid in AUTHORIZED_UIDS:
               print(f"✅ Access Granted [UID: {scanned_uid}]")
               print("Waiting for Rotation gesture...")
               ser.write(b'W')  # Tell Arduino to pulse red LED
               system_state = "WAIT_MOTION"
           else:
               print(f"❌ Access Denied [UID: {scanned_uid}]")

           scanned_uid = None
           history_x.clear()
           history_y.clear()

       # 3. Read MPU6050 Data
       data = ser.readline().decode('utf-8', errors='ignore').strip()
       if data:
           values = data.split(',')

           if len(values) == 3:
               try:
                   ax_val, ay_val, az_val = int(values[0]), int(values[1]),
int(values[2])
```

```
                    history_x.append(ax_val)
                    history_y.append(ay_val)
                    history_z.append(az_val)

                    plot_x.append(ax_val)
                    plot_y.append(ay_val)

                    if len(plot_x) > 100:
                        plot_x.pop(0)
                        plot_y.pop(0)

                    line.set_data(plot_x, plot_y)
                    plt.draw()
                    plt.pause(0.01)

                    # 4. Analyze Motion ONLY if we are in the WAIT_MOTION
state
                    if len(history_x) == window_size:
                        if system_state == "WAIT_MOTION":
                            motion_type = classify_motion(history_x,
history_y, history_z)

                            if "Idle" not in motion_type:
                                if "Rotation" in motion_type:
                                    print("🔄 Rotation verified! Opening door
for 5 seconds.")
                                    ser.write(b'R')
                                    system_state = "DOOR_BUSY"
                                    action_time = current_time
                                else:
                                    print(f"❌ Incorrect gesture
({motion_type}). Try again.")

                        history_x.clear()
                        history_y.clear()
                        history_z.clear()

                except ValueError:
                    pass
    except KeyboardInterrupt:
        print("\nProgram terminated.")
        break

ser.close()
```

1.4 DATA COLLECTION & ANALYSIS

1.4.1 Data Collection

The data was gathered by watching the physical hardware, looking at the real-time graph, and reading the software logs.

List of Authorized UIDs:

- UID 1: 0013020304 (Cytron Tag)
- *(Any unauthorized tag scanned was automatically rejected by the system)*

Observation Table:

Observation Phase	RFID Input (UID)	Sensor Movement (MPU6050)	Python Logic Evaluation	Hardware Status (LED)	Hardware Status (Servo Angle)
1. Initial State	None	Stationary	System Locked	Solid Red	0°
2. Authorized Scan	0013020304	Stationary	Waiting for Motion	Pulsing Red	0°
3. Invalid Gesture	N/A	Random Shake	Gesture Rejected	Pulsing Red	0°
4. Valid Gesture	N/A	Circular Motion	Rotation Verified	Solid Green	180°
5. Timer Hold	N/A	Stationary	5-Second Timer Active	Solid Green	180°
6. Auto-Lock Reset	N/A	Stationary	Timer Expired (Reset)	Solid Red	0°

1.4.2 Data Analysis

The collected data from the observation table confirmed several key technical achievements in the system design:

1. **Stable Bidirectional Communication:** The serial connection on COM5 at 9600 baud remained stable throughout the experiment. This continuous two-way connection was necessary because the Arduino had to constantly send raw sensor data (Ax, Ay, Az) to the computer while simultaneously waiting to receive the byte command ('R') to actuate the door.

2. **Background Threading Efficiency:** The threading logic in Python worked exactly as planned. Because the USB RFID scanner acts like a keyboard input, it would normally pause the entire program until a card is scanned. By putting the RFID listener on a background thread, the main program was able to keep plotting the live motion graph without freezing.

3. **Motion Detection Accuracy:** The Python math logic successfully filtered out random shaking and accurately recognized true circular gestures. It achieved this by first subtracting the average sensor value to remove the constant pull of gravity. Then, it checked if the movement variance was large enough to be an intentional gesture, and finally used 2D cross-product math to verify that the data points were rotating continuously in one direction.

4. **Non-Blocking Hardware Control:** Using a `millis()` timer in the Arduino code instead of a standard `delay()` function was crucial for the system's success. It allowed the Arduino to pulse the red LED and hold the servo motor open for exactly 5000 milliseconds without pausing the microcontroller. This ensured the sensor data kept flowing back to Python without overflowing the serial buffer or crashing the computer graph.

1.5 RESULTS & DISCUSSION

1.5.1 Results

The final task successfully proved that the hardware and software can work together for a two-step security system.

1. The system correctly ignored unregistered RFID tags and only started the motion tracking when the correct UID was scanned.
2. The Python code recognized the circular movements quickly.
3. The Arduino reacted immediately to the Python commands, moving the servo and changing the LEDs exactly as planned.

1.5.2 Discussion

Meaning of the Results: The experiment showed that it is better to let a computer do the heavy thinking. While an Arduino can do simple checks, the complex math needed to tell a circle from a straight line is much easier to write and run in Python.

Problems and Fixes:

- **Sensor Tilting:** If the MPU6050 was held at a tilt, gravity would pull on one axis and mess up the readings. This was fixed by subtracting the average value from the data in Python, which acted like an automatic calibration.
- **Code Freezing:** At first, using the delay() function in the Arduino code to keep the door open caused the sensor data to stop sending. This crashed the Python graph. This problem was completely solved by using a millis() timer instead.

1.6 CONCLUSION

This laboratory project successfully achieved its goal of creating a dual-layer smart access system using an Arduino and Python. By combining a physical RFID scanner with an MPU6050 motion sensor, the system demonstrated how hardware and software can work together to solve complex mechatronic tasks. The experiment proved that transferring heavy calculations to a computer is highly effective. Python easily handled the cross-product math needed for gesture recognition and managed the RFID inputs in the background without freezing the system. At the same time, the Arduino successfully acted as the physical controller by using non-blocking timers to move the servo motor and switch the LEDs.

During the session, practical knowledge was gained regarding sensor calibration, signal filtering, and state machine programming. Solving real problems like sensor gravity tilting and serial port freezing provided valuable hands-on troubleshooting experience. Ultimately, this project shows how basic electronic components can be integrated into responsive and secure access systems, providing a strong foundation for future human-machine interface designs.

1.7 RECOMMENDATIONS

1. **Use Fast Fourier Transform (FFT):** Python's numpy library could be used to do an FFT on the data. This would detect the circular motion based on frequencies instead of just looking at the data points, which would make the system better at ignoring random noise.

2. **Better Data Buffering:** The code currently clears the list of data points after a gesture is detected. Writing a proper circular buffer would stop any memory issues and make sure only the newest continuous data is checked, making the system respond even faster.

REFERENCES

Arduino. (2026). *Arduino language reference*. https://www.arduino.cc/reference/en/

Department of Mechatronics Engineering. (2026). *Laboratory manual for mechatronics system integration*. International Islamic University Malaysia.

Liechti, C. (2020). *pySerial's documentation: Welcome to pySerial*. https://pyserial.readthedocs.io/

Margolis, M., Jepson, B., & Weldin, N. R. (2020). *Arduino cookbook: Recipes to begin, expand, and enhance your projects* (3rd ed.). O'Reilly Media.

Random Nerd Tutorials. (n.d.). *Security access using MFRC522 RFID reader with Arduino*. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

APPENDICES

Video link on this experiment : https://drive.google.com/drive/folders/1-QWUq-JMn534ZMPMi1aAf1h91HjGrXIC?usp=sharing

ACKNOWLEDGEMENT

First and foremost, a sincere thank you goes to the lecturer for the constant guidance during this lab session. The clear explanations made connecting the Arduino code to the physical breadboard much easier to figure out.

Appreciation is also given to the Mechatronics Engineering department at IIUM for providing a great lab environment. Having access to all the right microcontrollers and electronic parts was a huge help in getting the digital counter to work properly.

A special mention goes to the lab technicians as well. Their quick help with checking the hardware and replacing faulty jumper wires saved a lot of time and kept the experiment moving.

Finally, a huge thanks goes out to the lab partners and classmates. Spending hours troubleshooting the circuit, sharing ideas, and fixing coding errors together made this a really good learning experience. The great teamwork is what made this whole project a success.

STUDENTS DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature :  Read [/]

Name : Muhammad Danish Fitri Bin Mohd Sharifuddin Understand [/]

Matric No : 2413559 Agree [/]

Signature :  Read [/]

Name : Muhammad Izz Zikry Bin Azwari Understand [/]

Matric No : 2218899 Agree [/]

Signature : 

Read [/]

Name : Samer Y. I. Lubbad

Understand [/]

Matric No : 2213959

Agree [/]

Signature : 

Read [/]

Name : Muhammad Faris Izhan Bin Johari

Understand [/]

Matric No : 2413501

Agree [/]